

Marine Ecosystem Simulation

Software Design Document

Killian Embler

04/15/2026

CS 225, Spring 2026

Embry-Riddle Aeronautical University

Daytona Beach campus

1 Aerospace Boulevard

Daytona Beach, FL 32114

Introduction:

This project took the form of a marine ecosystem simulator via a genetic algorithm. By using the differences between species classes and their evolutionary behavior and interactions with other species in a predetermined environment, the program simulated real world biodiversity and modeled the process of forming balanced / unbalanced marine ecosystems. Researchers may evaluate how the ecosystem responds to the addition or removal of various species and how the inclusion or removal of other species in return may serve to restore the delicate balance of an effective ecosystem. The application enabled the user to select species from a large file-stored directory, as well as contribute to the directory themselves, and introduce or remove a population of these selected species to a simulated ecosystem. They may then run the progression of time to see if the ecosystem is able to maintain balance between the species or if it will collapse and die out. At the end of each time frame, the user may make changes to the population of each species, view the entire state of the environment, inspect a certain specimen, or save the program to a file and exit. This is an exceptionally important resource for marine researchers in the modern era as human factors have been greatly accelerating the rate of conditional change in marine ecosystems via the endangerment or extinction of keystone species, environmental destruction, and the introduction of invasive species via human transportation methods. Ecosystem simulations and genetic algorithms are able to help consider what species may best adapt to these sudden changes and which ones may struggle to adapt in time and face extinction. Beyond this, many ecosystems may not be able to handle the loss of a keystone species or introduction of an invasive one, and simulations such as this may serve as early indication or warning systems for when the total collapse of an ecosystem is of urgent concern and direct human intervention becomes necessary. This document explores the complexities that arise in the development of large simulations such as this and how the program effectively models the real behavior of a marine ecosystem at scale while maintaining fidelity.

Problem Description:

Problem / Background:

As was previously mentioned in the introduction, marine ecosystems around the globe have been subjected to massively accelerated changes in recent years. This can be a result of several factors, notably human influence, changing climates, the extinction of keystone species, and the introduction of invasive species, all of which can cause great damage or even complete collapse in a delicately balanced ecosystem. This fast acting and high magnitude threat creates a significant need among researchers to create a predictive model capable of simulating an ecosystem's response to external stimuli.

This model was developed via application of a genetic algorithm. These algorithms take large populations of objects with a set of attributes and randomly increase or decrease these attributes over time while determining fitness. Objects with low fitness are culled, while those with high fitness are allowed to duplicate, which eventually drives the population via the statistical principle of natural selection towards an optimal state which is dependent upon the fitness criteria. This is an exact analog to the way that ecosystems and the organisms they contain face selection pressures which determine their reproductive success. While this general algorithmic concept can apply to many fields of evolutionary computation, this project approaches a genetic algorithm in a very literal sense, simulating the natural phenomena from which it was derived in the first place. This directness of application makes the technique extremely suited to modeling an ecosystem's response to external stimuli.

The primary object of concern in this project is that of a marine ecosystem. In order to model the behavior of these ecosystems, the project first seeks to effectively define a marine ecosystem where possible and approximate it otherwise. There are six key processes that take place in real world ecosystems that this project places emphasis on: environments, trophic levels, predator/prey dynamics, keystone species / invasive species / trophic cascades, ecosystem balance / collapse, and selective adaptation. An environment is essentially the "setting" of an ecosystem, in which organisms interact and compete against each other under conditions and pressures imposed by the environment itself. Trophic levels are what distinguishes between species in terms of producers, primary consumers (prey), and higher order consumers (predators) and establishes what a certain species role in the environment primarily is. The predator prey dynamic is defined as a "predation check" an event that can occur where one organism will attempt to consume another as necessary to survive, wherein the success of the predator means the death of the prey and vice versa. Keystone species, invasive species, and trophic cascades are what the model seeks to monitor and determine the influence of given time in an ecosystem, as they are crucial to the three primary modes of ecosystem collapse. Keystone species are species that, when absent, removed, or in low number leave a critical role unfilled and can destroy the balance of an ecosystem. Invasive species are those which are added to a stable ecosystem and are unfairly advantaged because the other present organisms have not had time to properly adapt to the presence of the new organisms. This is often of massive detriment to the native organisms and the ecosystem as a whole. It is important to note what it means for an ecosystem to "collapse" in the first place. To quantify this phenomenon, there are two established metrics which can be assessed: a biodiversity index and a producer index. It is a well-established concept in natural sciences that ecosystems that are greater in biodiversity - possessing a greater number of species with well-rounded population division - are more resilient and less likely to have species become naturally endangered or extinct, and the index serves to represent this quality numerically. In every ecosystem on earth, the trophic ladder always begins with producers, microorganisms or plants which obtain energy not from other organisms but from sunlight or geochemical sources. However, the population of primary consumers can have a massive impact on the population and health of an ecosystem's producers which, when either excessively high or low, can serve as an indicator signal for imminent

population collapse or surge for primary consumers, which can easily destabilize entire ecosystems and cause extinction events. Fortunately, nature has a response to the destabilization of ecosystems: selective adaptation - the process by which organisms with greater suitability under a set of selective pressures are more likely to pass their genes down to offspring and popularize the traits among the general population given generations passing. As time passes, the organisms in the environment will adapt to continue as a species, which, while mostly beneficial to the biodiversity and longevity of an environment, can also cause detrimental changes if a species becomes too dominant or a niche is no longer able to be filled. This reiterates the importance of the model as it can help researchers predict what kind of adaptation may occur under different circumstances and understand how to encourage or discourage this adaptation for the benefit of the ecosystem.

This project falls into the well documented category of agent-based ecological modeling, which is supported by a good amount of existing literature. These kinds of models are commonly used for practical purposes in ecological sciences and have been stated by the National Oceanic and Atmospheric Administration (NOAA) to be a key resource in supporting the health of all kinds of ecosystems. The closest analog to the model that was developed in this project is the open-source agent-based model published in October 2025 called *SwimmingIndividuals*, which models biological processes and adaptive responses with an emphasis upon modeling individual organisms. The application of ABM and a genetic algorithm in this kind of modeling is imperative because many of the adaptive responses have an inherent degree of randomness in them and cannot be evaluated analytically and must instead be modeled to the highest degree of fidelity possible and repeated to consider various outcomes and their probability. This type of application first began in 1975 when early genetic algorithms were first established with the intention of modeling ecological processes.

-----[**DISCLAIMER**]-----

The following material will examine the mechanics and mathematics behind the production of the marine ecosystem model. There are several key pieces of information, such as base stats, balancing constants, stat increases, and even demo species selections which will be detailed. The choice to include these as they are in their current state has been made by me to give what I believe will be a good view of multiple marine ecosystems and the interactions the model should be able to simulate. These decisions have been made with respect to my own knowledge of foundational marine ecological concepts and organisms which I find particularly interesting, as well as consulting with my mother, a marine biologist, on topics such as base stats and preferred prey. There is, however, some degree of arbitrary selection involved in this process, and some changes may need to be made to constants or default organisms in order to properly “balance” the model after a longer period of testing and experimentation with the model.

Mechanics and Functionality: Turn Structure

The simulation advances in units of time referred to as turns, each of which represents approximately one month of real-world elapsed time. During each turn, a fixed sequence of processes is executed for every organism in the environment in a set order. This ordering is critical, as the outcome of earlier processes directly determines the population available for later ones. The full turn sequence is defined in Table 1 below.

Step	Process	Description
1	Age Increment	Every organism in the environment has its age incremented by one turn.
2	Natural Death Check	Each organism undergoes an age-based death check and a depth tolerance check. Organisms that fail either check are removed from the environment.
3	Producer Status Update	Each primary consumer individually reduces producer status via its turn action. Producer status is then replenished by the environment recovery rate.
4	Producer Death Check	Each primary consumer undergoes an additional death check scaled by current producer status. Low producer status increases the probability of starvation death.
5	Predation	Each predatory organism selects a target from the available prey population weighted by preferred prey status and size proximity. A success check is then resolved. A successful predation results in the death of the prey organism.
6	Reproduction	Each organism undergoes a reproduction check scaled by its reproductivity stat and maturity factor. Organisms that pass the check produce one offspring with slightly redistributed genetic points plus 0-5 additional points.
7	Biodiversity Index Update	The Shannon Diversity Index is recalculated based on current species populations. Species with fewer than 10 individuals are excluded from the calculation.
8	User Interaction	The simulation pauses and presents the user with the interaction menu.

Table 1 - Simulation Turn Order

This order ensures that natural survival from aging and environmental pressure is resolved before predation and reproduction are evaluated, which reflects the reality that only living, healthy organisms participate in predation and reproduction events. The biodiversity index is recalculated at the end of each turn so that the value presented to the researcher always reflects the most current state of the ecosystem.

Mechanics and Functionality: User Interaction and Menu System

The simulation is operated entirely through a console / text based interactive menu system. The user is never required to interact with input files directly because all modifications to the simulation and the species directory are performed through the menu. The interaction system is divided into two phases: the launch sequence, which runs once at the beginning of each session, and the main interaction menu, which repeats after every action until the user chooses to save and exit.

Launch Sequence

Upon launching the program, the user is guided through a brief startup sequence before the main menu becomes available. This sequence determines whether the session will begin with a new simulation or resume from a previously saved state. The full launch sequence is outlined in Table 2 below.

Step	Prompt	Details
1	New or Load	The user is asked whether they wish to start a new simulation or load an existing one from a save file.
2a	Select Environment (New)	If starting a new simulation, the user selects from the three available environment types: Coral Reef, Open Ocean, or Deep Sea Trench. The environment object is then initialized with the appropriate parameters.
2b	Enter Save File Name (Load)	If loading an existing simulation, the user provides the name of the save file. The program reconstructs the full simulation state including all organism objects and environmental parameters from the file before proceeding.
3	Main Menu	The main interaction menu is presented. The user may now interact with the simulation.

Table 2 - Launch Sequence

If a new simulation is selected, the user chooses from one of three base environment types. Each environment type initializes with a distinct set of parameters including depth range, producer maximum, and recovery rate, as defined in Table 5. Once the environment is initialized, the main menu is presented right away, and the user may begin populating the environment with species from the directory before advancing time for the first turn.

If a save file is loaded, the program reads the header block to restore all environment parameters and then reconstructs each organism object from the following lines of the file. Once reconstruction is complete, the main menu is presented with the simulation state exactly as it was at the time of saving, including the turn number, producer status, and full organism population.

Main Interaction Menu

Following the launch sequence, the main interaction menu is presented to the user after every action. The menu remains available until the user selects the save and exit option. A summary of all available options is provided in Table 3, which is followed by a full description of each.

#	Option	Description
1	Add Organisms	Adds a specified number of organisms of a chosen species to the environment.
2	Remove Organisms	Removes a specified number of organisms of a chosen species from the environment.
3	View Species Stats	Displays the current population count and average genetic stat distribution for a single specified species.
4	View All Species Stats	Displays the current population count and average genetic stat distribution for every species present in the environment.
5	Sample Specimens	Randomly selects and displays the full individual stat spread and age of a user-specified number of organisms from a chosen species.
6	Define New Species	Guides the user through a set input process to define a new species and write it to the species directory file.
7	Advance Time	Advances the simulation by a user-specified number of turns, executing the full turn sequence for each.
8	Save and Exit	Writes the current simulation state and updated directory to save files in the current folder and ends the program.

Table 3 - Main Interaction Menu Options

Option 1: Add Organisms

The user is prompted to enter the Latin name of a species as it appears in the species directory. If the species is found in the directory, the user is then prompted to enter the number of organisms of that species to add to the environment. The program creates that number of new organism objects using the base stat distribution as defined in the directory, with each organism initialized at age zero. If the total organism count would exceed the environment maximum of 100,000 individuals, the program warns the user and adds only as many organisms as the remaining capacity allows.

This option is the primary mechanism by which researchers can introduce species into the simulation, including the introduction of invasive species and the possible deliberate introduction of natural predators to counteract an existing invasive population.

Option 2: Remove Organisms

The user is prompted to enter the Latin name of a species present in the environment and a number of individuals to remove. The program selects the number of organisms of the specified species from the environment's population and removes them. If the removal count exceeds the current population of that species, all remaining individuals of that species are removed and the user is notified. This option allows researchers to simulate the removal or endangerment of a species (likely to represent human influence), including the deliberate removal of a keystone species to observe the resulting trophic cascade.

Option 3: View Species Stats

The user is prompted to enter the Latin name of a species present in the environment. The program then displays information for that species:

- Current living population within the environment.
- The average value of each of the eight genetic stats across all living individuals of that species.
- The average total point pool across all living individuals of that species.

This option allows researchers to track the drift of a species' average stat distribution over time, making the effects of selective adaptation directly observable at a wider scale.

Option 4: View All Species Stats

The program displays the same information as Option 3 for every species present in the environment with at least one living individual. This provides a full picture of the current ecosystem state and allows researchers to assess the population health and evolutionary progression of all species simultaneously. The current biodiversity index value and producer index value are also displayed at the end of this output as general ecosystem health indicators.

Option 5: Sample Specimens

The user is prompted to enter the Latin name of a species present in the environment and a sample size N. The program randomly selects N individual organisms from that population and displays the complete individual stat spread and current age of each selected organism. If the requested sample size exceeds the current population, all individuals are displayed and the user is notified. An example of the output format for a single individual is shown below:

Specimen 1 of 5 - *Carcharhinus amblyrhynchos*

Age: 14

Size: 43

Speed: 37

Hunting: 40

Evasion: 21

Reproductivity: 13

Longevity: 32

Depth Tolerance: 29

Specialization: 38

Total Points: 253

This option is particularly valuable for observing the degree of genetic change within a species' population after many generations have passed. A researcher may compare sampled individuals against the original base stat distribution from the directory to assess how the population has adapted to its environment over the course of the simulation.

Option 6: Define New Species

The user is guided through a sequential series of prompts to define a new species, which is then written as a new entry to the species directory file and becomes available for use in the current and all future sessions. The full series of prompts and their valid input ranges are defined in Table 4 below.

Step	Field	Description	Valid Input
1	Latin Name	The unique species identifier. Must not already exist in the directory.	String
2	Common Name	The human-readable name of the species.	String

Step	Field	Description	Valid Input
3	Type	Defines whether the organism is a predator or primary consumer.	PREDATOR / PRIMARY_CONSUMER
4	Size	Starting size stat. Contributes to effective size and predation checks.	Integer ≥ 1
5	Speed	Starting speed stat. Affects pursuit and escape during predation.	Integer ≥ 0
6	Hunting	Starting hunting stat. Offensive predation success modifier. 0 for primary consumers.	Integer ≥ 0
7	Evasion	Starting evasion stat. Defensive survival modifier.	Integer ≥ 0
8	Reproductivity	Starting reproductivity stat. Controls duplication frequency.	Integer ≥ 1
9	Longevity	Starting longevity stat. Controls natural lifespan and maturity threshold.	Integer ≥ 1
10	Depth Tolerance	Starting depth tolerance stat. Controls survivability at environment depth.	Integer ≥ 0
11	Specialization	Starting specialization stat. Scales preferred prey bonus/penalty. 0 for primary consumers.	Integer ≥ 0
12	Prey Count	Number of preferred prey species (0-5). 0 for primary consumers.	Integer 0-5
13+	Preferred Prey 1-5	Latin name of each preferred prey species, entered one at a time. Must exist in the directory.	String (Latin name)

Table 4 - New Species Definition Prompts

If the entered Latin name already exists in the directory the program notifies the user and returns to the main menu without writing to the file. Upon successful completion of all prompts the new species is written to the directory object in the standard space-delimited format and the user is returned to the main menu. The updated directory object is saved to the directory file when the user saves and exits the program.

This option is central to the simulation's value as a research tool, as it allows marine researchers to introduce real or hypothetical species into the directory with biologically informed stat distributions and observe their interactions with the existing ecosystem population.

Option 7: Advance Time

The user is prompted to enter the number of turns to advance. The program then executes the full turn sequence once for each requested turn in succession, updating all organism states, producer status, and the biodiversity index with each pass. At the conclusion of the final turn the updated environment state and current biodiversity index are displayed and the user is returned to the main menu. If producer status reaches less than 25% at any point during the advance the user is notified that a producer collapse event occurred.

Option 8: Save and Exit

The user is prompted to enter a file name at which to write the save file. The program then writes the simulation state to that path in the save file format described in the File Processing section, including the header block with current environment parameters and one line per living organism. Upon successful completion of the write operation the program displays a confirmation message and exits. If the specified file path is invalid the user will be re-prompted for a new file name. This option is available at any point during the main menu interaction and is not restricted to any particular phase of the turn sequence.

Mechanics and Functionality: Predation System

The predation system is the most mechanically complex component of the simulation and is responsible for modeling the competitive interactions between predatory organisms and their prey. Every predatory organism initiates one predation attempt per turn. This process consists of two sequential phases: target selection and a success check.

Effective Size

Before any predation calculation is performed, the effective size of each organism is computed. Rather than using raw size stats, which represent an organism at full physical maturity, effective size scales the size stat by the organism's current age relative to its longevity stat in order to account for the reduced physical capacity of juvenile organisms:

$$EffectiveSize = Size \times (Age / Longevity)$$

This ratio will fall below 1.0 for juvenile organisms and will exceed 1.0 for those which have surpassed their natural lifespan. As natural death probability rises sharply past full longevity, few organisms in practice will reach a significantly elevated effective size, though the rare individual may represent the real-world phenomenon of an exceptionally large specimen of a given species.

Target Selection

Each predatory organism selects a target from the pool of all other living organisms in the environment. The probability of any given organism being selected as a target is determined by a selection weight, which accounts for two factors: whether the potential prey belongs to the predator's preferred prey list, and the size difference between the predator and the potential prey.

$$\text{SelectionWeight} = (1 + \text{PreferredBonus}) \times e^{(-0.003 \times |\text{EffectiveSizePredator} - \text{EffectiveSizePrey}|^2)}$$

The PreferredBonus is nonzero only when the potential prey species appears on the predator's preferred prey list, in which case it is computed as:

$$\text{PreferredBonus} = \text{Specialization} / \text{NumberOfPreferredPrey}$$

This scaling ensures that a highly specialized predator with a single preferred prey item receives a large selection bonus toward that species, while a more generalist predator with five preferred prey items receives a smaller but still meaningful bonus distributed across all of them. The exponential size penalty term causes the selection weight to fall off rapidly as the size difference between predator and prey increases in either direction, reflecting the real-world tendency of predators to target prey of a manageable and worthwhile size relative to themselves. The constant $k = 0.003$ was chosen such that a size difference of 20 points reduces selection weight by approximately 30%, and a difference of 40 points reduces it by approximately 80%.

Predation Success Check

Once a target has been selected, a success check is resolved to determine whether the predation attempt results in a kill. The outcome has two possibilities: a successful check results in the immediate death and removal of the prey organism, while a failed check results in the death and removal of the predator. The success probability is computed as follows:

$$\text{SuccessChance} = [(\text{Hunting} + \text{Speed})\text{Predator} / ((\text{Hunting} + \text{Speed})\text{Predator} + (\text{Evasion} + \text{Speed})\text{Prey} \times 0.1)] \times \text{SpecMod} \times \text{SizePenalty}$$

The 0.1 divisor applied to the prey's defensive stats intentionally skews the outcome in favor of the predator, reflecting the asymmetry present in real predator-prey encounters where a single successful hunt results in the death of the prey while a single failed hunt is merely a missed opportunity for the predator.

The SpecMod term applies the specialization modifier based on whether the selected prey is a preferred prey species:

$$\text{SpecMod} = 1 + (\text{Specialization} / \text{NumberOfPreferredPrey}) \quad [\text{if preferred prey}]$$

$$\text{SpecMod} = 1 - (\text{Specialization} / \text{StatMax}) \quad [\text{if not preferred prey}]$$

This means that a highly specialized predator which has managed to find its preferred prey will enjoy a substantial success bonus, while that same predator hunting outside its specialization will be penalized proportionally. This mechanic is central to the simulation's ability to model invasive species dynamics, as a newly introduced species with no native predators will face only unspecialized hunters operating at a penalty until the ecosystem has time to adapt.

The SizePenalty term in the success check uses the same exponential curve as in target selection, applied once more to penalize success checks where the size difference between predator and prey is large in either direction.

Mechanics and Functionality: Natural Death and Aging

Each turn, prior to predation and reproduction, every organism undergoes two independent death checks: one based on age relative to longevity, and one based on the compatibility of the organism's depth tolerance with the current environment. Either check resolving in death results in the organism's removal from the environment.

Age-Based Death

The probability of natural death due to aging is governed by a polynomial curve that remains low throughout most of an organism's lifespan and rises sharply as the organism approaches and exceeds its longevity stat:

$$AgeDeathChance = (Age / (Longevity \times 1.5))^4$$

The factor of 1.5 in the denominator shifts the steep rise of the curve to 1.5 times the organism's longevity stat, meaning that organisms routinely survive to their full longevity value but rarely far beyond it. The exponent of 4 ensures the curve remains nearly flat for young and middle-aged organisms before rising sharply, which is consistent with the biological reality that most organisms in a healthy state face very low natural mortality until late in their lifespan.

Depth Tolerance Death

Each environment defines a characteristic depth range. If an organism's depth tolerance stat is lower than the environment's depth value, it faces an additional death probability each turn proportional to the magnitude of the mismatch:

$$DepthDeathChance = \max(0, (EnvironmentDepth - DepthTolerance) / DepthTolerance)$$

This linear relationship means that small mismatches between an organism and its environment impose a modest but meaningful additional mortality, while large mismatches rapidly become lethal. This mechanic is particularly significant for cross-environment species which may be introduced into environments at the edge of their natural depth range, and for invasive species whose depth tolerance may not be well suited to their introduced environment.

Mechanics and Functionality: Reproduction and Genetic Drift

Reproduction is the mechanism by which the genetic algorithm advances across generations. Each turn, after death checks and predation have been resolved, every surviving organism undergoes a reproduction check. If the check succeeds, the organism produces one offspring. This asexual reproduction model is a deliberate simplification of biological reality, analogous to the parthenogenetic and budding reproduction strategies present in a number of real marine species and is employed here to maintain simulation simplicity without sacrificing the essential dynamics of generational genetic drift.

Maturity and Reproduction Probability

An organism cannot reproduce until it has reached sexual maturity, which is modeled as a fixed fraction of its longevity stat. The maturity threshold is defined as:

$$MaturityAge = Longevity \times 0.2$$

Below this age the organism has zero reproduction probability regardless of its reproductivity stat. Above this age, the probability scales smoothly upward using both a maturity factor and the organism's reproductivity stat:

$$MaturityFactor = \max(0, (Age - MaturityAge) / (Longevity - MaturityAge))$$

$$ReproChance = MaturityFactor \times (Reproductivity / (Reproductivity + 40))$$

The second term of this equation is a curve that prevents reproduction probability from ever reaching 100% regardless of how high the reproductivity stat grows over generations. The constant 40 was chosen such that a reproductivity stat of 50 yields a greater than 50% base reproduction rate at full maturity, which produces biologically plausible generational turnover rates across the range of species in the base directory.

Genetic Drift on Duplication

When an organism successfully reproduces, its offspring inherits the parent's full genetic stat distribution as a baseline. Two modifications are then applied. First, a small portion of the organism's total point pool is randomly redistributed among its eight stats:

$$RedistributedPoints = \text{round}(TotalPoints \times 0.01)$$

Second, a random integer between 0 and 5 additional points is added to the offspring's total pool, distributed randomly across its stats. These additional points represent the accumulation of minor genetic improvements over generations and are the primary driver of long-term evolutionary divergence within a species population. Over many generations, organisms in environments that strongly favor certain stats will see those stats drift upward in the population, while less useful stats may stagnate or decline due to redistribution. It is important to note that preferred prey relationships are defined at the species level and are not subject to genetic drift; only the specialization stat, which governs the magnitude of the preferred prey bonus, may change over generations.

Mechanics and Functionality: Producer Status

Producer status is a scalar attribute of the environment representing the population and health of the microscopic producers at the base of the marine food web, including phytoplankton, algae, and other photosynthetic or chemosynthetic microorganisms. It is expressed as a value between 0 and a maximum defined per environment type, referred to as ProducerMax. Producer status serves as the foundational resource upon which all primary consumers depend and its decline is the primary driver of bottom-up ecosystem collapse.

Recovery

At the beginning of the producer update phase each turn, producer status recovers naturally by an amount proportional to a recovery rate defined by the environment type:

$$ProducerRecovery = ProducerMax \times EnvironmentRecoveryRate$$

The recovery rate varies between environment types to reflect the differing productivity of real marine ecosystems. Shallow, nutrient-rich environments such as coral reefs support far more vigorous producer growth than the sparse, low-light conditions of deep sea trenches. The recovery rates and ProducerMax values for each environment in the base set are defined in Table 5 below.

Environment	Depth Range	Producer Max	Recovery Rate	Favored Stats
Coral Reef	0-50m	20,000	0.75% / turn	Evasion, Speed
Open Ocean (Pelagic)	0-200m	15,000	0.5% / turn	Speed, Hunting
Deep Sea Trench	200-1000m	10,000	0.25% / turn	Depth Tolerance, Size

Table 5 - Environment Definitions and Producer Parameters

Consumption

Following recovery, producer status is reduced by the cumulative grazing activity of all primary consumer organisms currently living in the environment. Each primary consumer reduces producer status proportionally to its effective size each turn, reflecting the greater resource consumption of larger individuals:

$$\text{GrazingMod} = -(\text{EffectiveSize} / 500)$$

Each primary consumer individually applies this modifier to the environment's producer status via `modifyProducerStatus()` during its turn action. The scaling factor of 500 ensures that consumption scales naturally with effective size, with larger and older organisms consuming proportionally more resources.

Starvation Mortality

When producer status falls below its maximum, primary consumers face an additional probability of death due to resource scarcity each turn, applied during the producer death check phase of the turn sequence:

$$\text{ProducerDeathChance} = \max(0, 1 - (\text{ProducerStatus} / \text{ProducerMax}))^5 \times 0.95$$

At full producer status this term evaluates to zero and imposes no additional mortality. As producer status declines toward zero the additional death probability approaches 0.95, meaning that at complete producer depletion every primary consumer faces a 95% additional chance of death each turn on top of their natural aging mortality. Producer status can reach zero if primary consumer populations are large enough and persistent enough to outpace the environment's recovery rate, at which point the starvation mortality will rapidly drive primary consumer populations downward. If the primary consumer population declines sufficiently before total collapse, producer status may begin to recover, demonstrating the self-regulating feedback loop that characterizes real trophic dynamics.

Mechanics and Functionality: Biodiversity Index

The biodiversity index is the primary quantitative metric by which the health and stability of the simulated ecosystem is assessed. It is computed using the Shannon Diversity Index, a well-established measure in ecological science that accounts for both the number of distinct species present in an ecosystem and the evenness with which the total population is distributed among them. An ecosystem with many species each represented by a healthy and proportionate population will yield a high index value, while an ecosystem dominated by one or two species or experiencing widespread extinction will yield a low one.

The index is computed each turn following the completion of all death, predation, and reproduction events. Only species with a living population of 10 or more individuals are considered extant for the purposes of this calculation; species falling below this threshold are treated as functionally absent from the ecosystem. This threshold is defined here as a recommended interpretive guideline rather than a hard-coded simulation rule, and researchers are encouraged to apply their own judgment when evaluating index values in the context of specific research scenarios.

The Shannon Diversity Index is computed as follows:

$$H = -\sum (p_i \times \ln(p_i))$$

Where p_i is the proportion of the total living population belonging to species i , and the sum is taken over all species with a population of 10 or more.

Mechanics and Functionality: Systems Inputs and Outputs

The simulation accepts input from two sources: the user via an interactive console menu, and the file system via the species directory and simulation save files. Output is delivered to the user via console display and optionally written back to the file system as a save file. The full set of user-facing menu interactions available each turn is as follows:

- Add organisms of a specified species to the environment by entering the species' Latin name and a desired population count.
- Remove organisms of a specified species from the environment by entering the species' Latin name and a removal count.
- View the average genetic stat distribution and total population of a specified species currently present in the environment.
- Display the general population and average stats for every species currently present in the environment.
- Inspect an individual organism specimen, displaying its full current stat spread and age.
- Define a new species and write it to the species directory file for use in future sessions.
- Advance the simulation forward by one turn, executing the full turn sequence described above.
- Save the current simulation state to a file and exit.

Upon launching the simulation, if no save file is being loaded, the user is first prompted to select an environment type from the three available base environments. The simulation then initializes the environment object with the appropriate parameters and presents the user with the interaction menu. If a save file path is provided at launch, the simulation instead reconstructs the full environment state from the file before presenting the menu.

Mechanics and Functionality: File Processing

The simulation interacts with two distinct plain text file types: the species directory file and the simulation save file. Both files use space-delimited formatting with underscores substituting for spaces within multi-word fields, as the space character serves as the field delimiter. Neither file is intended for direct human editing; all modifications to the species directory are performed through the in-program species definition menu.

Species Directory File

The species directory file stores the definition of every species available for use in the simulation. Each species occupies exactly one line in the file, with fields appearing in a fixed order:

*LATIN_NAME COMMON_NAME TYPE SIZE SPEED HUNTING EVASION
REPRODUCTIVITY LONGEVITY DEPTH_TOLERANCE SPECIALIZATION PREY_COUNT
[PREY_1 ... PREY_N]*

The TYPE field accepts exactly two values: PREDATOR or PRIMARY_CONSUMER. The PREY_COUNT field contains an integer between 0 and 5 indicating how many preferred prey Latin names follow on the same line. Primary consumer species will always have a PREY_COUNT of 0 and no subsequent prey fields. Latin names in the preferred prey list are stored using the same underscore substitution convention as all other multi-word fields. Two example lines from the base directory are shown below:

*Carcharhinus_amblyrhynchos Grey_Reef_Shark PREDATOR 40 35 38 20 12 30 28 35 3
Plectropomus_leopardus Cheilinus_undulatus Thalassoma_bifasciatum*

Stenopus_hispidus Banded_Coral_Shrimp PRIMARY_CONSUMER 4 12 0 18 38 12 32 0 0

The complete base species directory comprising all 50 species defined for this simulation is provided across Tables 6 through 8 below for reference. Table 6 defines species identity and classification, Table 7 defines the initial genetic stat distribution for each species, and Table 8 defines preferred prey relationships.

Common Name	Latin Name	Classification	Environment(s)
CORAL REEF			
Grey Reef Shark	<i>Carcharhinus amblyrhynchos</i>	Predator (Apex)	Coral Reef
Leopard Coral Grouper	<i>Plectropomus leopardus</i>	Predator	Coral Reef

Common Name	Latin Name	Classification	Environment(s)
Red Lionfish	<i>Pterois volitans</i>	Predator (INVASIVE)	Coral Reef
Devil Firefish	<i>Pterois miles</i>	Predator (INVASIVE)	Coral Reef
Sixline Wrasse	<i>Pseudocheilinus hexataenia</i>	Predator	Coral Reef
Day Octopus	<i>Octopus cyanea</i>	Predator	Coral Reef
Reef Stonefish	<i>Synanceia verrucosa</i>	Predator	Coral Reef
Humphead Wrasse	<i>Cheilinus undulatus</i>	Predator (Keystone)	Coral Reef
Zebra Dwarf Lionfish	<i>Dendrochirus zebra</i>	Predator	Coral Reef
Brown Tang	<i>Acanthurus nigrofuscus</i>	Primary Consumer	Coral Reef
Yellow Tang	<i>Zebrasoma flavescens</i>	Primary Consumer	Coral Reef
Long Spined Sea Urchin	<i>Diadema antillarum</i>	Primary Consumer (Keystone)	Coral Reef
Bluehead Wrasse	<i>Thalassoma bifasciatum</i>	Primary Consumer	Coral Reef
Banded Coral Shrimp	<i>Stenopus hispidus</i>	Primary Consumer	Coral Reef
Blue Starfish	<i>Linckia laevigata</i>	Primary Consumer	Coral Reef
OPEN OCEAN (PELAGIC)			
Shortfin Mako Shark	<i>Isurus oxyrinchus</i>	Predator (Apex)	Pelagic
Yellowfin Tuna	<i>Thunnus albacares</i>	Predator	Pelagic
Swordfish	<i>Xiphias gladius</i>	Predator	Pelagic
Mahi-Mahi	<i>Coryphaena hippurus</i>	Predator	Pelagic
Blue Shark	<i>Prionace glauca</i>	Predator	Pelagic
Humboldt Squid	<i>Dosidicus gigas</i>	Predator	Pelagic
Opah	<i>Lampris guttatus</i>	Predator	Pelagic
European Anchovy	<i>Engraulis encrasicolus</i>	Primary Consumer	Pelagic
Pacific Sardine	<i>Sardinops sagax</i>	Primary Consumer	Pelagic

Common Name	Latin Name	Classification	Environment(s)
Tropical Flying Fish	<i>Exocoetus volitans</i>	Primary Consumer	Pelagic
Emerald Rockcod	<i>Trematomus bernacchii</i>	Primary Consumer	Pelagic
Flathead Grey Mullet	<i>Mugil cephalus</i>	Primary Consumer	Pelagic
Green Sea Turtle	<i>Chelonia mydas</i>	Primary Consumer (Keystone)	Pelagic
Giant Oceanic Manta Ray	<i>Manta birostris</i>	Primary Consumer	Pelagic
DEEP SEA TRENCH			
Giant Squid	<i>Architeuthis dux</i>	Predator (Apex)	Deep Sea Trench
Sloane's Viperfish	<i>Chauliodus sloanei</i>	Predator	Deep Sea Trench
Humpback Anglerfish	<i>Melanocetus johnsonii</i>	Predator	Deep Sea Trench
Vampire Squid	<i>Vampyroteuthis infernalis</i>	Predator	Deep Sea Trench
Dumbo Octopus	<i>Grimpoteuthis boylei</i>	Predator	Deep Sea Trench
Boa Dragonfish	<i>Stomias boa</i>	Predator	Deep Sea Trench
Fangtooth Fish	<i>Anoplogaster cornuta</i>	Predator	Deep Sea Trench
Antarctic Krill	<i>Euphausia superba</i>	Primary Consumer (Keystone)	Deep Sea Trench
Giant Isopod	<i>Bathynomus giganteus</i>	Primary Consumer	Deep Sea Trench
Glass Shrimp	<i>Hymenodora glacialis</i>	Primary Consumer	Deep Sea Trench
Spotted Lanternfish	<i>Myctophum punctatum</i>	Primary Consumer	Deep Sea Trench
Sea Pig	<i>Scotoplanes globosa</i>	Primary Consumer	Deep Sea Trench
Bioluminescent Copepod	<i>Metridia lucens</i>	Primary Consumer	Deep Sea Trench
Red King Crab	<i>Paralithodes camtschaticus</i>	Primary Consumer	Deep Sea Trench
CROSS-ENVIRONMENT SPECIES			
Bottlenose Dolphin	<i>Tursiops truncatus</i>	Predator	Cross-Environment
California Sea Lion	<i>Zalophus californianus</i>	Predator	Cross-Environment

Common Name	Latin Name	Classification	Environment(s)
Loggerhead Sea Turtle	<i>Caretta caretta</i>	Predator	Cross-Environment
Sperm Whale	<i>Physeter macrocephalus</i>	Predator (Apex)	Cross-Environment
Northern Elephant Seal	<i>Mirounga angustirostris</i>	Predator	Cross-Environment
Great White Shark	<i>Carcharodon carcharias</i>	Predator (Apex)	Cross-Environment

Table 6 - Species Directory: Identity and Classification

Common Name	Size	Speed	Hunting	Evasion	Repro.	Longevity	Depth Tol.	Spec.	Total
CORAL REEF									
Grey Reef Shark	40	35	38	20	12	30	28	35	238
Leopard Coral Grouper	28	28	30	22	18	25	30	32	213
Red Lionfish	22	20	32	30	28	20	25	18	195
Devil Firefish	20	20	30	30	28	20	25	18	191
Sixline Wrasse	10	25	22	28	30	18	30	38	201
Day Octopus	18	22	28	32	20	15	32	35	202
Reef Stonefish	20	8	38	25	15	28	35	40	209
Humphead Wrasse	35	20	28	22	12	35	30	38	220
Zebra Dwarf Lionfish	12	15	25	28	22	20	30	35	187
Brown Tang	10	20	-	25	30	18	30	-	133
Yellow Tang	8	22	-	28	32	16	30	-	136
Long Spined Sea Urchin	8	3	-	15	25	22	35	-	108
Bluehead Wrasse	6	28	-	30	35	14	30	-	143
Banded Coral Shrimp	4	12	-	18	38	12	32	-	116
Blue Starfish	6	2	-	8	20	28	35	-	99

Common Name	Size	Speed	Hunting	Evasion	Repro.	Longevity	Depth Tol.	Spec.	Total
OPEN OCEAN (PELAGIC)									
Shortfin Mako Shark	45	48	40	18	10	32	22	38	253
Yellowfin Tuna	35	42	32	25	18	25	22	30	229
Swordfish	42	38	36	20	12	28	22	35	233
Mahi-Mahi	30	40	28	25	20	20	22	32	217
Blue Shark	38	30	30	20	15	30	22	28	213
Humboldt Squid	25	30	30	28	25	15	25	25	203
Opah	32	25	25	22	15	28	22	28	197
European Anchovy	5	25	-	28	40	10	22	-	130
Pacific Sardine	6	22	-	25	38	12	22	-	125
Tropical Flying Fish	10	35	-	32	28	14	20	-	139
Emerald Rockcod	15	15	-	20	22	22	25	-	119
Flathead Grey Mullet	12	20	-	22	28	18	22	-	122
Green Sea Turtle	30	15	-	18	8	45	20	-	136
Giant Oceanic Manta Ray	48	18	-	15	6	38	20	-	145
DEEP SEA TRENCH									
Giant Squid	50	28	35	22	10	35	48	32	260
Sloane's Viperfish	15	25	35	22	20	22	45	38	222
Humpback Anglerfish	18	10	38	18	15	25	48	40	212
Vampire Squid	12	18	22	30	18	28	50	30	208
Dumbo Octopus	15	15	25	25	18	25	48	32	203
Boa Dragonfish	12	20	30	22	20	20	45	35	204

Common Name	Size	Speed	Hunting	Evasion	Repro.	Longevity	Depth Tol.	Spec.	Total
Fangtooth Fish	10	18	32	20	22	18	45	38	203
Antarctic Krill	3	15	-	20	48	8	42	-	136
Giant Isopod	14	5	-	12	15	32	48	-	126
Glass Shrimp	4	18	-	22	35	12	45	-	136
Spotted Lanternfish	5	20	-	25	38	12	40	-	140
Sea Pig	6	3	-	8	25	20	50	-	112
Bioluminescent Copepod	2	12	-	18	50	6	42	-	130
Red King Crab	20	5	-	15	12	35	45	-	132
CROSS-ENVIRONMENT SPECIES									
Bottlenose Dolphin	28	38	30	28	18	28	18	25	213
California Sea Lion	30	35	28	25	15	25	18	22	198
Loggerhead Sea Turtle	32	15	25	18	8	42	18	30	188
Sperm Whale	55	20	38	15	5	50	38	40	261
Northern Elephant Seal	40	25	28	20	10	30	30	25	208
Great White Shark	52	35	42	18	8	38	20	35	248

Table 7 - Species Directory: Genetic Stat Distribution

Common Name	Classification	Preferred Prey
CORAL REEF		
Grey Reef Shark	Predator (Apex)	<i>Plectropomus leopardus</i> , <i>Cheilinus undulatus</i> , <i>Thalassoma bifasciatum</i>
Leopard Coral Grouper	Predator	<i>Acanthurus nigrofuscus</i> , <i>Zebrasoma flavescens</i> , <i>Stenopus hispidus</i>

Common Name	Classification	Preferred Prey
Red Lionfish	Predator (INVASIVE)	<i>Thalassoma bifasciatum, Stenopus hispidus, Acanthurus nigrofuscus</i>
Devil Firefish	Predator (INVASIVE)	<i>Thalassoma bifasciatum, Stenopus hispidus, Zebrasoma flavescens</i>
Sixline Wrasse	Predator	<i>Pterois volitans, Pterois miles, Stenopus hispidus</i>
Day Octopus	Predator	<i>Stenopus hispidus, Linckia laevigata, Acanthurus nigrofuscus</i>
Reef Stonefish	Predator	<i>Acanthurus nigrofuscus, Thalassoma bifasciatum</i>
Humphead Wrasse	Predator (Keystone)	<i>Diadema antillarum, Linckia laevigata</i>
Zebra Dwarf Lionfish	Predator	<i>Stenopus hispidus, Thalassoma bifasciatum</i>
Brown Tang	Primary Consumer	N/A - Primary Consumer
Yellow Tang	Primary Consumer	N/A - Primary Consumer
Long Spined Sea Urchin	Primary Consumer (Keystone)	N/A - Primary Consumer
Bluehead Wrasse	Primary Consumer	N/A - Primary Consumer
Banded Coral Shrimp	Primary Consumer	N/A - Primary Consumer
Blue Starfish	Primary Consumer	N/A - Primary Consumer
OPEN OCEAN (PELAGIC)		
Shortfin Mako Shark	Predator (Apex)	<i>Thunnus albacares, Coryphaena hippurus, Lampris guttatus</i>
Yellowfin Tuna	Predator	<i>Engraulis encrasicolus, Sardinops sagax, Exocoetus volitans</i>
Swordfish	Predator	<i>Dosidicus gigas, Engraulis encrasicolus, Sardinops sagax</i>
Mahi-Mahi	Predator	<i>Exocoetus volitans, Engraulis encrasicolus</i>
Blue Shark	Predator	<i>Dosidicus gigas, Sardinops sagax, Engraulis encrasicolus</i>
Humboldt Squid	Predator	<i>Engraulis encrasicolus, Sardinops sagax, Mugil cephalus</i>
Opah	Predator	<i>Dosidicus gigas, Exocoetus volitans</i>
European Anchovy	Primary Consumer	N/A - Primary Consumer

Common Name	Classification	Preferred Prey
Pacific Sardine	Primary Consumer	N/A - Primary Consumer
Tropical Flying Fish	Primary Consumer	N/A - Primary Consumer
Emerald Rockcod	Primary Consumer	N/A - Primary Consumer
Flathead Grey Mullet	Primary Consumer	N/A - Primary Consumer
Green Sea Turtle	Primary Consumer (Keystone)	N/A - Primary Consumer
Giant Oceanic Manta Ray	Primary Consumer	N/A - Primary Consumer
DEEP SEA TRENCH		
Giant Squid	Predator (Apex)	<i>Myctophum punctatum, Hymenodora glacialis, Grimpoteuthis boylei</i>
Sloane's Viperfish	Predator	<i>Myctophum punctatum, Hymenodora glacialis, Stomias boa</i>
Humpback Anglerfish	Predator	<i>Myctophum punctatum, Bathynomus giganteus, Hymenodora glacialis</i>
Vampire Squid	Predator	<i>Hymenodora glacialis, Metridia lucens, Scotoplanes globosa</i>
Dumbo Octopus	Predator	<i>Bathynomus giganteus, Hymenodora glacialis, Scotoplanes globosa</i>
Boa Dragonfish	Predator	<i>Myctophum punctatum, Hymenodora glacialis</i>
Fangtooth Fish	Predator	<i>Myctophum punctatum, Metridia lucens, Paralithodes camtschaticus</i>
Antarctic Krill	Primary Consumer (Keystone)	N/A - Primary Consumer
Giant Isopod	Primary Consumer	N/A - Primary Consumer
Glass Shrimp	Primary Consumer	N/A - Primary Consumer
Spotted Lanternfish	Primary Consumer	N/A - Primary Consumer
Sea Pig	Primary Consumer	N/A - Primary Consumer
Bioluminescent Copepod	Primary Consumer	N/A - Primary Consumer
Red King Crab	Primary Consumer	N/A - Primary Consumer
CROSS-ENVIRONMENT SPECIES		

Common Name	Classification	Preferred Prey
Bottlenose Dolphin	Predator	<i>Engraulis encrasicolus</i> , <i>Sardinops sagax</i> , <i>Mugil cephalus</i>
California Sea Lion	Predator	<i>Thunnus albacares</i> , <i>Mugil cephalus</i> , <i>Trematomus bernacchii</i>
Loggerhead Sea Turtle	Predator	<i>Stenopus hispidus</i> , <i>Linckia laevigata</i> , <i>Diadema antillarum</i>
Sperm Whale	Predator (Apex)	<i>Architeuthis dux</i> , <i>Dosidicus gigas</i>
Northern Elephant Seal	Predator	<i>Myctophum punctatum</i> , <i>Lampris guttatus</i> , <i>Dosidicus gigas</i>
Great White Shark	Predator (Apex)	<i>Tursiops truncatus</i> , <i>Zalophus californianus</i> , <i>Chelonia mydas</i>

Table 8 - Species Directory: Preferred Prey Relationships

Simulation Save File

The simulation save file preserves the complete state of a running simulation so that it may be closed and resumed in a future session. The file begins with a header block containing environment-level data, followed by one line per living organism containing that organism's individual current stats. The header block format is as follows:

```
TURN: 42 ENVIRONMENT: Coral_Reef PRODUCER_STATUS: 8450.00 PRODUCER_MAX: 20000.00
BIODIVERSITY_INDEX: 0.85 ORGANISM_COUNT: 3847
```

Following the header line, each subsequent line represents one living organism in the environment at the time of saving, with fields in another fixed order:

```
LATIN_NAME AGE SIZE SPEED EVASION REPRODUCTIVITY LONGEVITY
DEPTH_TOLERANCE SPECIALIZATION HUNTING
```

Organism type and preferred prey information are not stored in the save file, as these attributes are fixed at the species level and are retrieved from the species directory upon loading. This design ensures that the save file remains compact even for large simulations while retaining full fidelity of the individual genetic state of every organism. In a worst-case scenario of 100,000 organisms, a save file is estimated to occupy approximately 2 megabytes, which is well within the practical constraints of any modern file system.

Problem Solution:

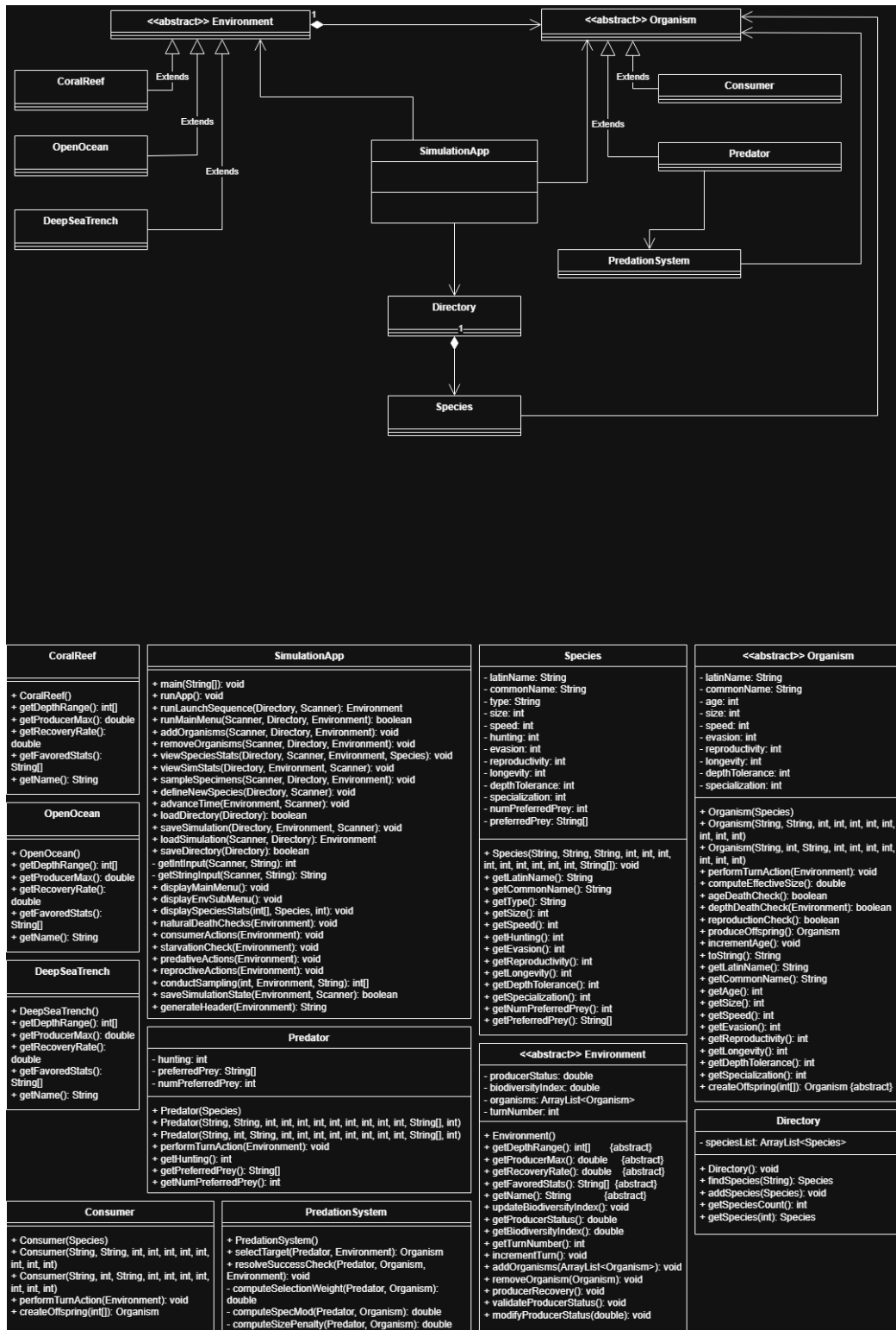


Figure 1: UML Class Diagram

The marine ecosystem simulation is built around eleven classes organized into two inheritance hierarchies, two auxiliary data structures, and two standalone classes. The design uses object-oriented programming to mirror the natural structure of the problem: environments are distinct from one another but share common behavior, organisms differ biologically but are all governed by the same core mechanics, and all simulation state and user interaction runs through a single central driver class.

SimulationApp

SimulationApp is the driver of the entire program. It handles all user interaction and file input/output and holds no persistent instance variables of its own. Instead, it creates a Directory and passes both it and the Environment as parameters between methods as needed.

- **main(String[])** - Instantiates SimulationApp and calls runApp() to start the program.
- **runApp()** - Creates the Scanner and Directory, calls runLaunchSequence() to set up the environment, then loops runMainMenu() until the user exits.
- **runLaunchSequence(Directory, Scanner): Environment** - Loads the species directory from file, then either loads a saved simulation or walks the user through selecting a new environment type, returning the result.
- **runMainMenu(Scanner, Directory, Environment): boolean** - Displays the main menu and routes the user's choice to the right method, returning true when the user selects Save and Exit.
- **addOrganisms(Scanner, Directory, Environment): void** - Asks the user for a species Latin name and a count, checks that adding them won't exceed the 100,000-organism limit, and creates the right number of Predator or Consumer instances.
- **removeOrganisms(Scanner, Directory, Environment): void** - Asks the user for a species and a count, caps the removal at however many of that species are currently alive, and removes them from the environment.
- **viewSpeciesStats(Directory, Scanner, Environment, Species): void** - Receives the Species object from the main menu, computes the living population and average stat values, and delegates display to displaySpeciesStats.
- **viewSimStats(Directory, Environment, Scanner): void** - Prints environment-level data including turn number, producer status, and biodiversity index, followed by population and average stats for every species with at least one living individual.
- **sampleSpecimens(Scanner, Directory, Environment): void** - Asks for a species and sample size, picks that many unique individuals at random, and prints each one's full stat spread and age using toString().
- **defineNewSpecies(Directory, Scanner): void** - Walks the user through a series of prompts to define a new species, validates all inputs including clamping negative preferred prey count to zero, and writes the finished entry to the directory file.

- **advanceTime(Environment, Scanner): void** - Asks the user how many turns to advance and runs the full turn sequence that many times: natural death checks, consumer actions, starvation checks, predation actions, reproductive actions, producer recovery, biodiversity update, and age increment.
- **loadDirectory(Directory): boolean** - Reads and parses directory.txt, fills the Directory with Species objects, and returns false if the file cannot be found.
- **saveSimulation(Directory, Environment, Scanner): void** - Writes the full current simulation state to a save file, including an environment header block followed by one line per living organism with all its current stats.
- **loadSimulation(Scanner, Directory): Environment** - Reads a save file specified by the user, rebuilds the environment and all organism objects from the stored data, handles malformed files gracefully via try-catch, and returns the restored Environment.
- **saveSimulationState(Environment, Scanner): boolean** - Writes the environment header via generateHeader() followed by one line per living organism in space-delimited format. Returns true on success.
- **saveDirectory(Directory): boolean** - Writes the current in-memory Directory back to directory.txt, preserving any species added during the session. Returns true on success.
- **getIntInput(Scanner, String): int** - Private helper that displays a prompt, reads a line, returns it as an integer, and rejects negative values.
- **getStringInput(Scanner, String): String** - Private helper that displays a prompt and returns the raw line from the scanner.
- **displayMainMenu(): void** - Prints the numbered main menu options to the console.
- **displayEnvSubMenu(): void** - Prints the environment type selection submenu.
- **displaySpeciesStats(int[], Species, int): void** - Formats and prints the stat averages and population count for a single species.
- **naturalDeathChecks(Environment): void** - Iterates all organisms and removes those that fail the age or depth death checks.
- **consumerActions(Environment): void** - Iterates all Consumer organisms and calls performTurnAction on each.
- **starvationCheck(Environment): void** - Computes cubic starvation probability per EQ8 and removes each Consumer that fails.
- **predativeActions(Environment): void** - Iterates all Predator organisms and calls performTurnAction on each.
- **reproctiveActions(Environment): void** - Iterates all organisms, runs reproductionCheck, and adds offspring to the environment.
- **conductSampling(int, Environment, String): int[]** - Performs random sampling and returns aggregated stat totals for average computation.
- **generateHeader(Environment): String** - Produces the save file header string.

Environment

Environment is an abstract class that defines the shared structure and behavior of all three simulation environments, maintaining producer status, biodiversity index, turn number, and the organism population as instance variables. The turn logic and producer update math that apply to every environment are implemented directly here, while the parameters that differ between environments are left as abstract methods for each subclass to fill in. The three concrete subclasses - CoralReef, OpenOcean, and DeepSeaTrench - each extend Environment and implement the five abstract methods with the values from Table 5, adding no new attributes of their own.

- **Environment()** - Sets producer status to ProducerMax, biodiversity index to zero, turn number to zero, and initializes an empty organism list.
- **getDepthRange(): int[] {abstract}** - Returns the minimum and maximum depth values for this environment.
- **getProducerMax(): double {abstract}** - Returns the maximum possible producer status for this environment type.
- **getRecoveryRate(): double {abstract}** - Returns the fraction of ProducerMax recovered each turn.
- **getFavoredStats(): String[] {abstract}** - Returns the names of the stats that are most advantaged by this environment.
- **getName(): String {abstract}** - Returns the display name of this environment type.
- **producerRecovery(): void** - Adds the per-turn recovery amount to producer status per EQ9. Grazing is handled individually by each Consumer during performTurnAction via modifyProducerStatus().
- **updateBiodiversityIndex(): void** - Recalculates the Shannon Diversity Index across all species with ten or more living individuals and stores the result.
- **incrementTurn(): void** - Adds one to the turn counter.
- **addOrganisms(ArrayList<Organism>): void** - Appends all organisms in the provided list to the environment's internal organism list.
- **removeOrganism(Organism): void** - Removes a specific organism instance from the internal list.
- **getProducerStatus(): double** - Returns the current producer status.
- **getBiodiversityIndex(): double** - Returns the current biodiversity index.
- **getTurnNumber(): int** - Returns the current turn number.
- **getOrganismCount(): int** - Returns how many organisms are currently in the environment.
- **getOrganism(int): Organism** - Returns the organism at the given index.
- **getSpeciesCount(String): int** - Returns how many living organisms match the given Latin name.

- **validateProducerStatus(): void** - Keeps producerStatus within [0, producerMax].
- **modifyProducerStatus(double): void** - Adds the given modifier to producerStatus, used by Consumer grazing.
- **getOrganism(int): Organism** - Returns the organism at the given index.

Organism

Organism is an abstract class that defines the structure and shared behavior of every individual in the simulation, holding nine genetic stats along with Latin name, common name, and age. Methods covering universal biological processes like death checks, reproduction, and offspring production are all implemented here, while the behaviors that differ between organism types are left abstract for subclasses to define. This lets the turn loop call performTurnAction on every organism using polymorphism and get the right behavior depending on whether the organism is a Predator or Consumer.

- **Organism(Species)** - Builds an organism from a Species template, copying all base stats and setting age to zero.
- **Organism(String, String, int, int, int, int, int, int, int)** - Builds an organism directly from individual stat values, used when creating offspring with redistributed stats.
- **Organism(String, int, String, int, int, int, int, int, int, int)** - Constructs an organism with age included, used when loading from a save file.
- **performTurnAction(Environment): void {abstract}** - Runs this organism's per-turn behavior; what exactly happens depends on whether it's a Predator or Consumer.
- **createOffspring(int[]): Organism {abstract}** - Builds and returns a new organism of the correct subtype from a given stat array, called by produceOffspring().
- **computeEffectiveSize(): double** - Returns size times the ratio of current age to longevity, per EQ1.
- **ageDeathCheck(): boolean** - Rolls against the EQ2 probability and returns true if the organism dies of age-related causes this turn.
- **depthDeathCheck(Environment): boolean** - Rolls against the EQ3 probability based on how far the organism's depth tolerance falls short of the environment's minimum depth.
- **reproductionCheck(): boolean** - Returns false if the organism hasn't reached maturity yet; otherwise rolls against the EQ4 probability based on maturity factor and reproductivity.
- **produceOffspring(): Organism** - Randomly redistributes 2.5% of the total stat pool, adds zero to five bonus points, then calls createOffspring() to build and return the result.
- **incrementAge(): void** - Adds one to the organism's age.
- **toString(): String** - Returns a formatted string of all stats, age, and total point pool for console display.

- **Getters for all instance variables** - getLatinName(), getCommonName(), getAge(), getSize(), getSpeed(), getEvasion(), getReproductivity(), getLongevity(), getDepthTolerance(), getSpecialization().

Predator

Predator extends Organism and represents all predatory species, adding three instance variables that only predators need: hunting, preferredPrey stored as a String array, and numPreferredPrey. Each turn a Predator creates a PredationSystem and uses it to select a target and resolve the hunt, keeping all the predation math out of this class.

- **Predator(Species)** - Builds a Predator from a Species template, passing base stats to the Organism constructor and pulling hunting, preferredPrey, and numPreferredPrey from the Species object.
- **Predator(String, String, int, int, int, int, int, int, int, int, String[], int)** - Direct-stat constructor used when producing predator offspring, carrying prey relationship fields over from the parent unchanged.
- **Predator(String, int, String, int, int, int, int, int, int, int, String[], int)** - Constructor with age, used when loading from a save file.
- **performTurnAction(Environment): void** - Creates a PredationSystem, calls selectTarget() to pick a prey organism, and passes both to resolveSuccessCheck() to determine what happens.
- **createOffspring(int[]): Organism** - Builds and returns a new Predator from the redistributed stat array, keeping hunting, preferredPrey, and numPreferredPrey the same as the parent.
- **getHunting(): int** - Returns the hunting stat.
- **getPreferredPrey(): String[]** - Returns the preferred prey array.
- **getNumPreferredPrey(): int** - Returns the number of preferred prey species.

Consumer

Consumer extends Organism and covers all primary consumer species, adding no new instance variables. Each turn a Consumer individually reduces the environment's producer status by calling modifyProducerStatus with a grazing modifier computed from its effective size per EQ7. Offspring production follows the same mechanism defined in Organism.

- **Consumer(Species)** - Builds a Consumer from a Species template, passing everything to the Organism constructor.
- **Consumer(String, String, int, int, int, int, int, int, int, int)** - Direct-stat constructor used when producing consumer offspring.

- **performTurnAction(Environment): void** - Computes the grazing modifier as negative effective size divided by 500 per EQ7 and applies it to the environment's producer status via modifyProducerStatus().
- **createOffspring(int[]): Organism** - Builds and returns a new Consumer from the redistributed stat array.

PredationSystem

PredationSystem holds all the predation logic from the Predation System section of this document, keeping the math isolated from the Predator class and in one place. It gets created fresh by each Predator during performTurnAction and exposes two public methods that handle the two phases of every predation attempt.

- **PredationSystem()** - Default constructor.
- **selectTarget(Predator, Environment): Organism** - Repeatedly samples organisms using weighted random selection, computing a weight per EQ5 for each candidate, until one is accepted and returned as the target.
- **resolveSuccessCheck(Predator, Organism, Environment): void** - Computes success probability per EQ6 using hunting, speed, evasion, SpecMod, and SizePenalty. On success the prey is removed from the environment; on failure the predator is removed.
- **computeSelectionWeight(Predator, Organism): double** - Private helper that combines the preferred prey bonus and size penalty into a single selection weight clamped between 0.0 and 1.0.
- **computeSpecMod(Predator, Organism): double** - Private helper that returns a boosted modifier if the candidate is preferred prey, or a penalized modifier if not, scaled by the predator's specialization stat.
- **computeSizePenalty(Predator, Organism): double** - Private helper that returns the exponential size-difference penalty used in both target selection and success resolution.

Species

Species is a data-only class that stores the base stats and identity information for one species as read from the directory file. Species objects are built once at load time and act as immutable templates that SimulationApp uses to construct Predator and Consumer instances; they are never changed after initialization.

- **Species(String, String, String, int, int, int, int, int, int, int, String[])** - Constructor accepting all identity and stat fields in directory file order.
- **Getters for all fields** - getLatinName(), getCommonName(), getType(), getSize(), getSpeed(), getHunting(), getEvasion(), getReproductivity(), getLongevity(), getDepthTolerance(), getSpecialization(), getNumPreferredPrey(), getPreferredPrey().

Directory

Directory holds the full set of Species objects available during the simulation and provides the lookup and insertion methods that SimulationApp uses throughout a session. It gets populated at launch by loadDirectory() and can grow if the user defines new species through the menu.

- **Directory()** - Initializes the internal ArrayList of Species objects.
- **findSpecies(String): Species** - Searches the list for a species whose Latin name matches the given string (case-insensitive) and returns it, or null if nothing matches.
- **getSpeciesCount(): int** - Adds a new Species object to the internal list.
- **addSpecies(Species): void** - Appends a Species object to the internal list.
- **getSpeciesCount(): int** - Returns how many species are currently in the directory.
- **getSpecies(int): Species** - Returns the Species at the given index.

References

- Holland, J. H. (1975). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. University of Michigan Press. (2nd ed., MIT Press, 1992)
- Hamblin, S. (2013). On the practical usage of genetic algorithms in ecology and evolution. *Methods in Ecology and Evolution*, 4(2), 184-194. <https://doi.org/10.1111/2041-210X.12000>
- Woodstock, M. S., Mattern, J. P., Wu, Z., & Britten, G. L. (2025). *SwimmingIndividuals: A high-performance agent-based model for marine ecosystems* [Preprint]. bioRxiv. <https://doi.org/10.1101/2025.10.13.681996>
- ** The above model is still pre-publication but notably similar so important to include**
- NOAA Fisheries. (n.d.). *Ecosystem modeling*. National Oceanic and Atmospheric Administration. <https://www.fisheries.noaa.gov/national/ecosystems/ecosystem-modeling>
- Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3-4), 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Hagen, O., Fluck, B., Fopp, F., Cabral, J. S., Hartig, F., Pontarp, M., Rangel, T. F., & Pellissier, L. (2021). gen3sis: A general engine for eco-evolutionary simulations of the processes that shape Earth's biodiversity. *PLOS Biology*, 19(7), e3001340. <https://doi.org/10.1371/journal.pbio.3001340>